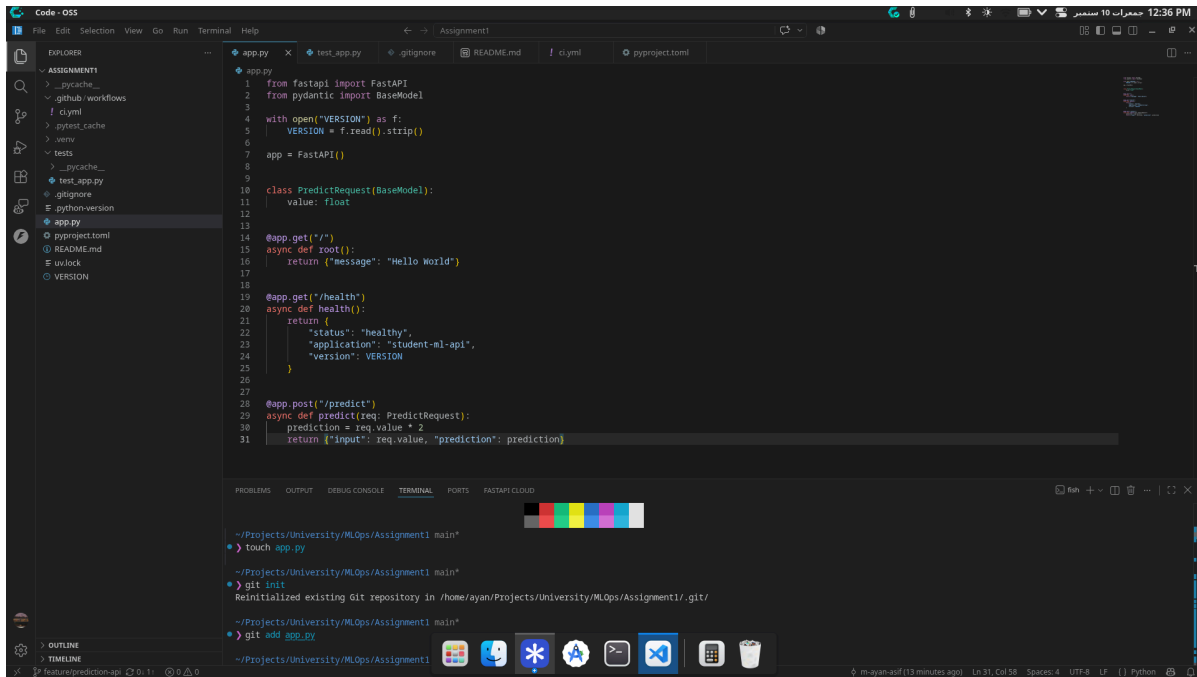


MLOps Assignment 1

M. Ayan Asif - 22i-1097 - CS B

Part 1 - Application with /health and /predict

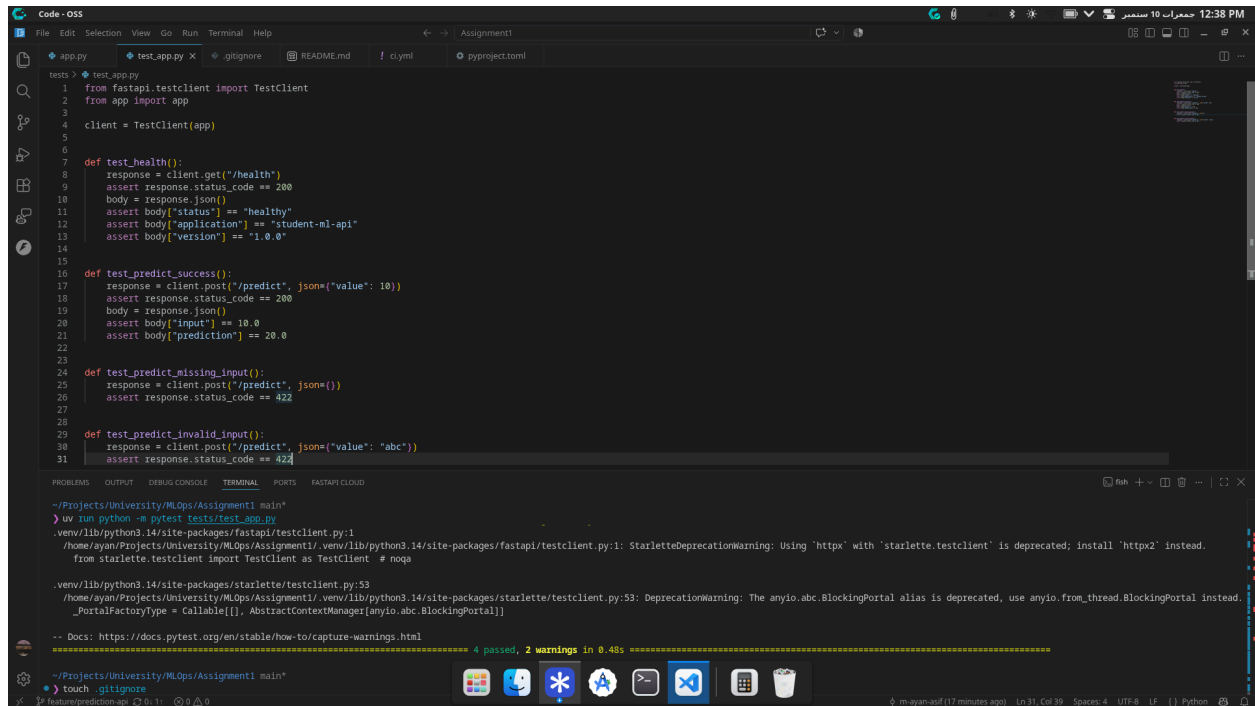


```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3
4 with open("VERSION") as f:
5     VERSION = f.read().strip()
6
7 app = FastAPI()
8
9 class PredictRequest(BaseModel):
10     value: float
11
12
13 @app.get("/")
14 async def root():
15     return {"message": "Hello World"}
16
17
18 @app.get("/health")
19 async def health():
20     return {
21         "status": "healthy",
22         "application": "student-ml-api",
23         "version": VERSION
24     }
25
26
27 @app.post("/predict")
28 async def predict(req: PredictRequest):
29     prediction = req.value * 2
30     return {"input": req.value, "prediction": prediction}
```

```
~/Projects/University/MLops/Assignment1 main*
➤ touch app.py
~/Projects/University/MLops/Assignment1 main*
➤ git init
Reinitialized existing Git repository in /home/ayan/Projects/University/MLops/Assignment1/.git/
~/Projects/University/MLops/Assignment1 main*
➤ git add app.py
```

- FastAPI app (`app.py`) with `/health` and `/predict` endpoints
- `/health` returns status/application/version (version read from `VERSION` file)
- `/predict` accepts `{"value": <number>}`, returns `{"input", "prediction"}`, formula is `value * 2`

Part 2: Automated tests



The screenshot shows a VS Code editor window with a file named `test_app.py` open. The file contains four test functions: `test_health()`, `test_predict_success()`, `test_predict_missing_input()`, and `test_predict_invalid_input()`. Each function uses `TestClient` to interact with an application and asserts the response status and body. The terminal at the bottom shows the command `uv run python -m pytest tests/test_app.py` being executed. The output indicates that all four tests passed, with a total of 4 passed and 2 warnings. The warnings are related to deprecated `Starlette` and `anyio` APIs. The status bar at the bottom shows the current file is `test_app.py` and the project is `Assignment1`.

```
1 from fastapi.testclient import TestClient
2 from app import app
3
4 client = TestClient(app)
5
6
7 def test_health():
8     response = client.get("/health")
9     assert response.status_code == 200
10    body = response.json()
11    assert body["status"] == "healthy"
12    assert body["application"] == "student-ml-api"
13    assert body["version"] == "1.0.0"
14
15
16 def test_predict_success():
17     response = client.post("/predict", json={"value": 10})
18     assert response.status_code == 200
19     body = response.json()
20     assert body["input"] == 10.0
21     assert body["prediction"] == 20.0
22
23
24 def test_predict_missing_input():
25     response = client.post("/predict", json={})
26     assert response.status_code == 422
27
28
29 def test_predict_invalid_input():
30     response = client.post("/predict", json={"value": "abc"})
31     assert response.status_code == 422
```

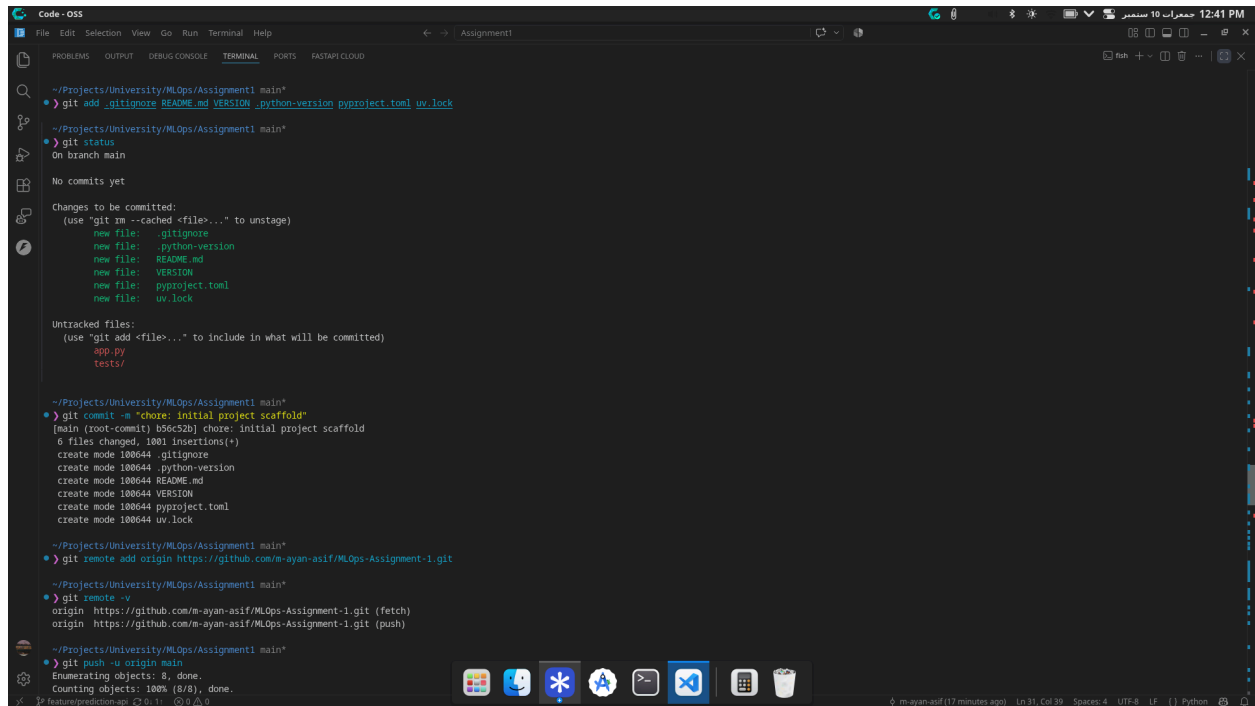
```
~/Projects/University/MLops/Assignment1 main*
> uv run python -m pytest tests/test_app.py
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1
/home/ayan/Projects/University/MLops/Assignment1/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa
.venv/lib/python3.14/site-packages/starlette/testclient.py:53
/home/ayan/Projects/University/MLops/Assignment1/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 4 passed, 2 warnings in 0.48s =====

~/Projects/University/MLops/Assignment1 main*
> touch .gitignore
```

- `tests/test_app.py` with 4 tests: health check, successful predict, missing input, invalid input
- All passing locally via `uv run pytest -v`

Part 3: Git workflow

A screenshot of a Visual Studio Code terminal window with a dark theme. The terminal shows the execution of several Git commands in a directory named ~/Projects/University/MLops/Assignment1. The commands and their outputs are as follows:
1. `git add .gitignore README.md VERSION .python-version pyproject.toml uv.lock`
2. `git status`
Output: "On branch main", "No commits yet", "Changes to be committed: (use 'git rm --cached <file>...' to unstage)", a list of 6 new files, and "Untracked files: app.py, tests/".
3. `git commit -m "chore: initial project scaffold"`
Output: "[main (root-commit) b56c5b] chore: initial project scaffold", "6 files changed, 1001 insertions(+)", and a list of created files and modes.
4. `git remote add origin https://github.com/m-ayan-asif/MLops-Assignment-1.git`
5. `git remote -v`
Output: "origin https://github.com/m-ayan-asif/MLops-Assignment-1.git (fetch)", "origin https://github.com/m-ayan-asif/MLops-Assignment-1.git (push)".
6. `git push -u origin main`
Output: "Enumerating objects: 8, done.", "Counting objects: 100% (8/8), done.".
The terminal window includes a sidebar with icons for Explorer, Search, Run and Debug, and Source Control. The top status bar shows the file path, line and column numbers, and the active language (Python).

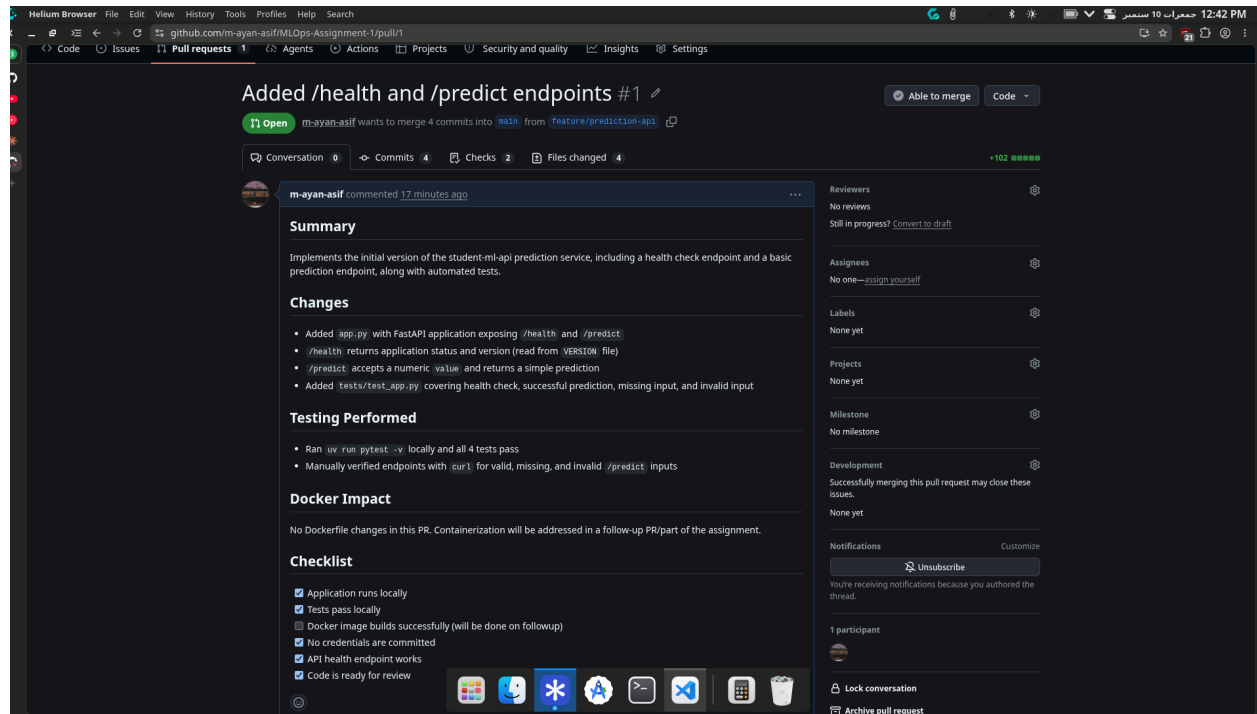
main initialized with scaffold commit only (.gitignore, README.md, VERSION, pyproject.toml, uv.lock, .python-version)

feature/prediction-api branch created off main

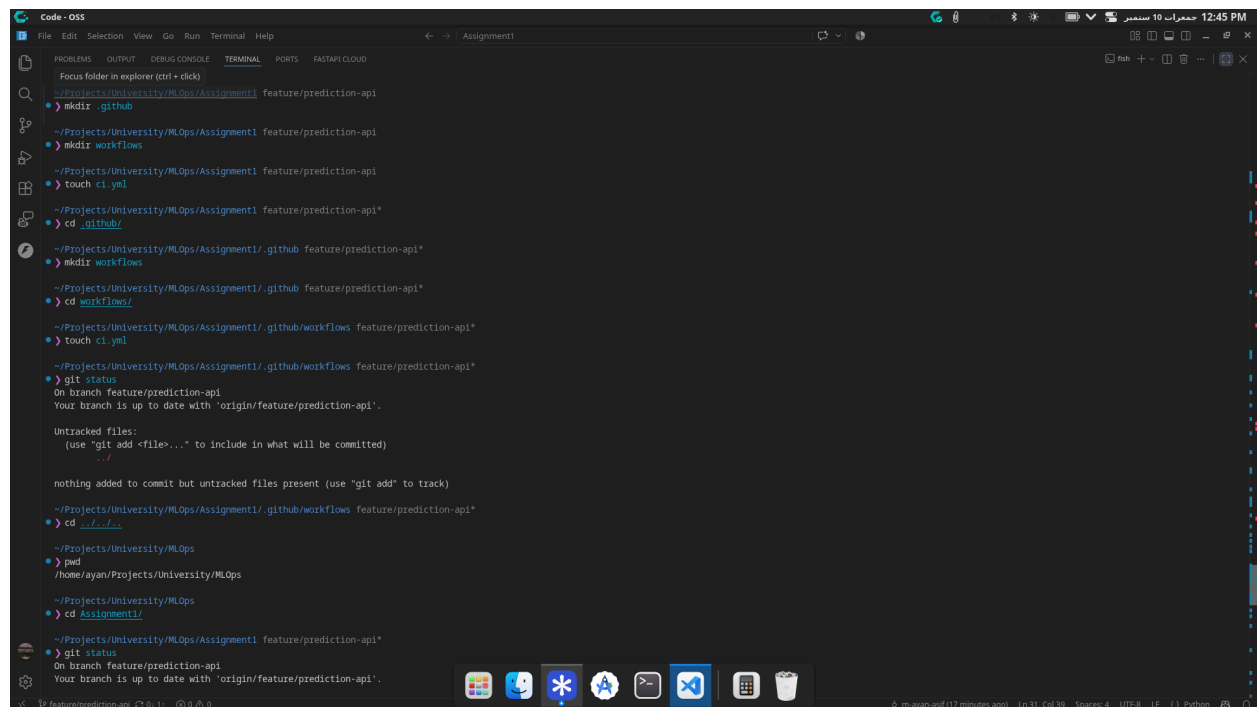
Two meaningful commits on the feature branch: feat: add health and predict endpoints, test: add API unit tests

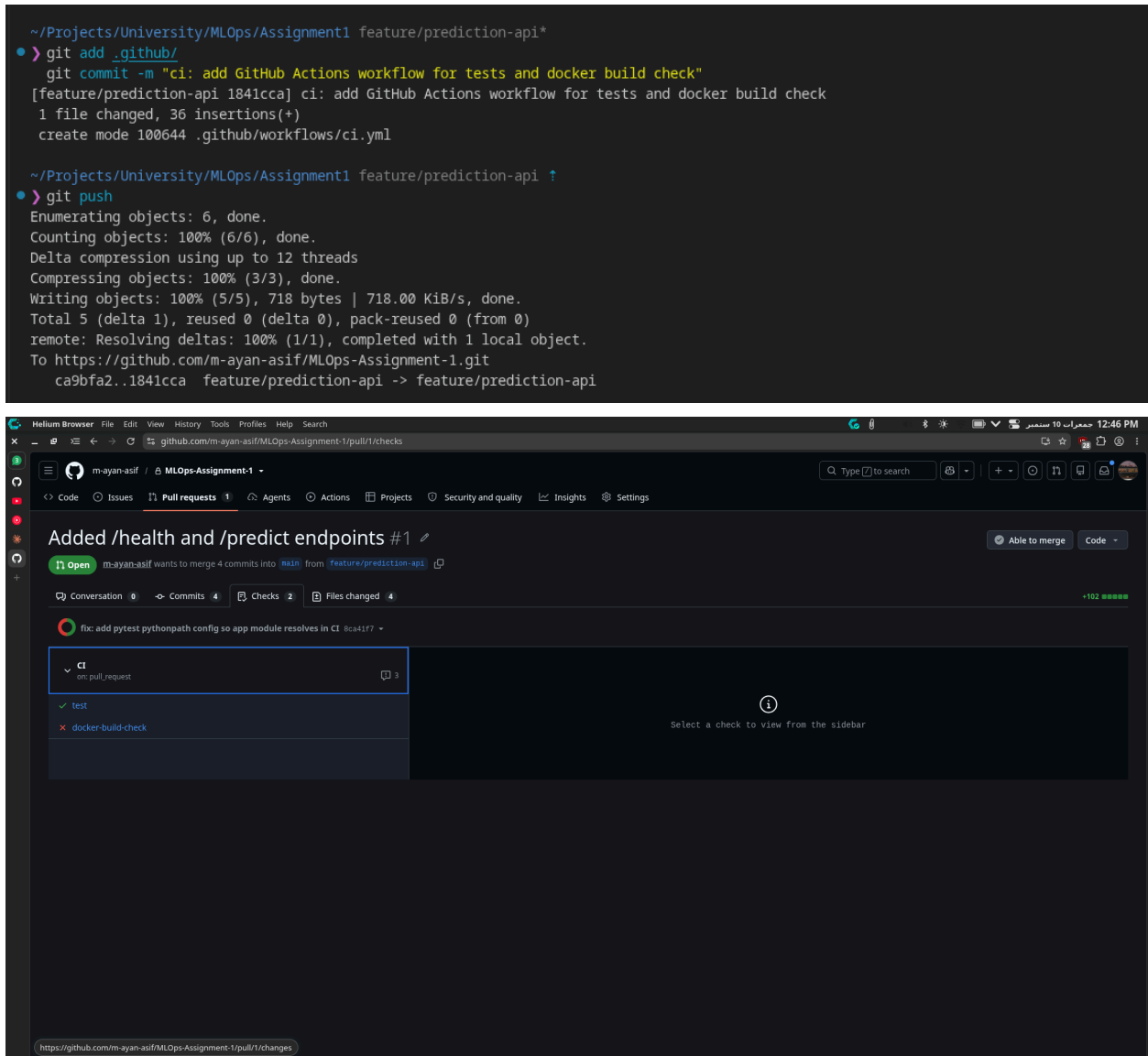
Pushed to GitHub, remote origin set up

Part 4: PR

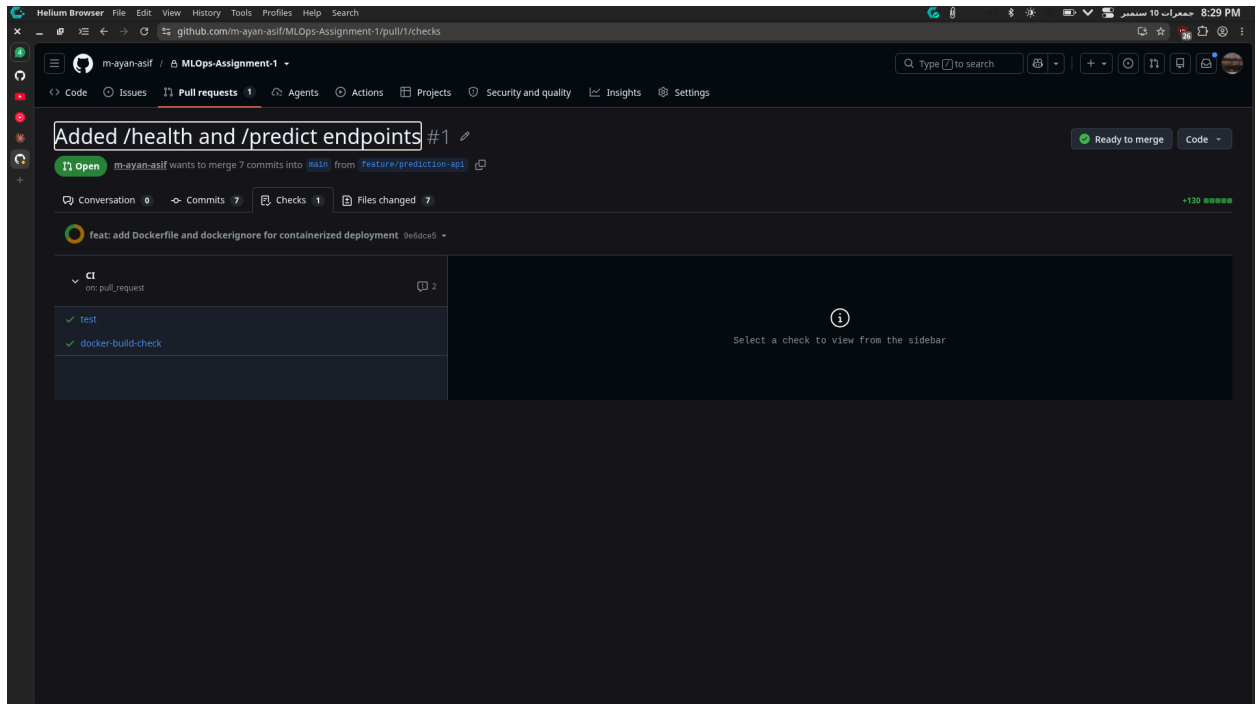


Part 5: Github Actions (partially)



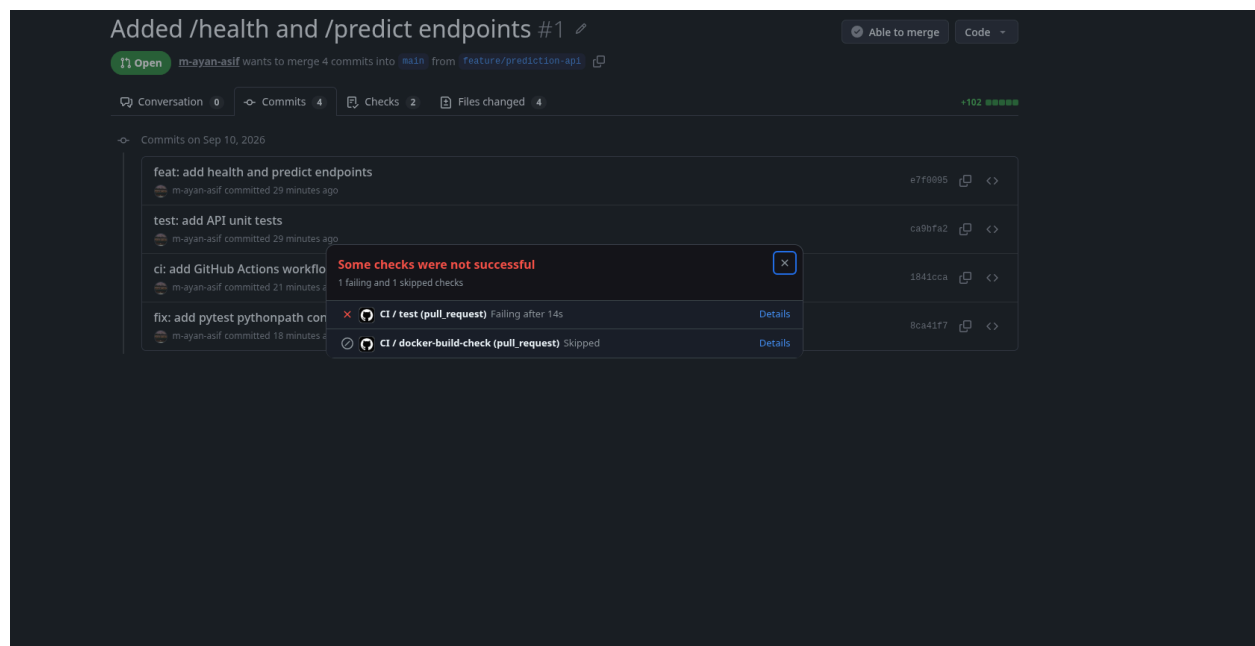


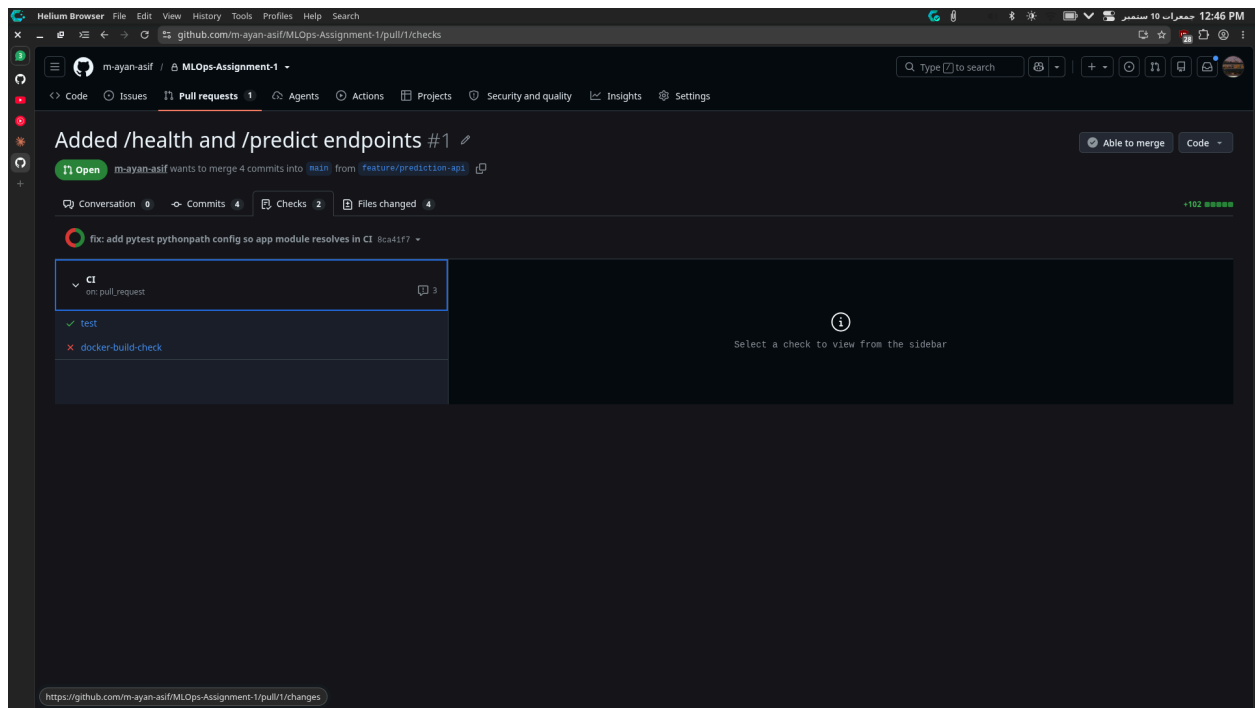
- `.github/workflows/ci.yml` created with two jobs: `test` (pytest) and `docker-build-check` (depends on `test`)
- CI runs automatically on PR → `main`
- Currently fails as Dockerfile not setup (fulfilling part 6 requirements)



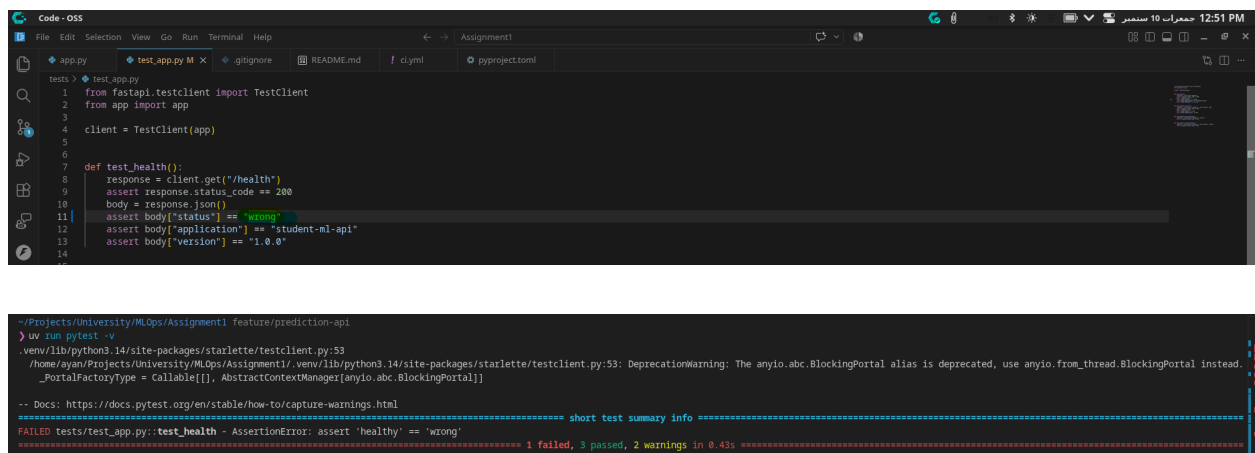
After doing step 9 and 10.

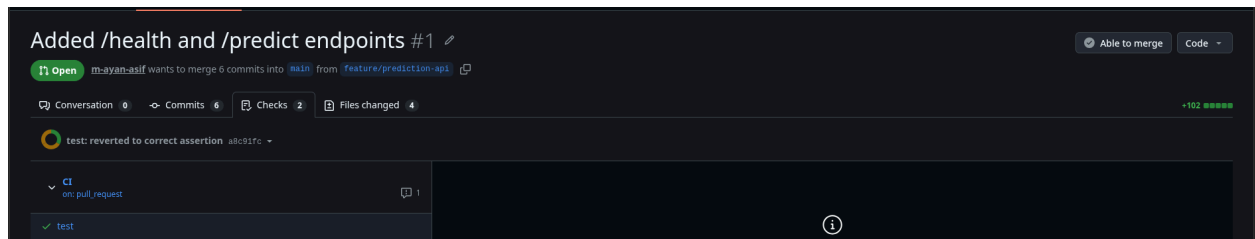
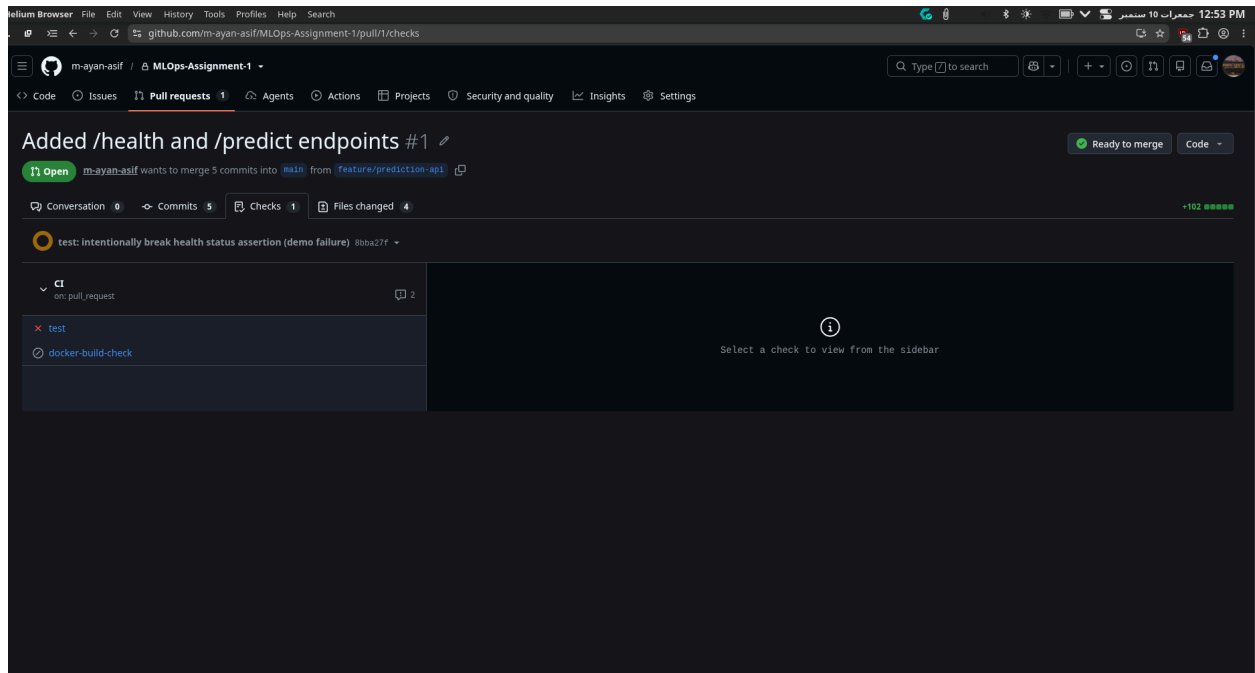
Part 6: Deliberate failure





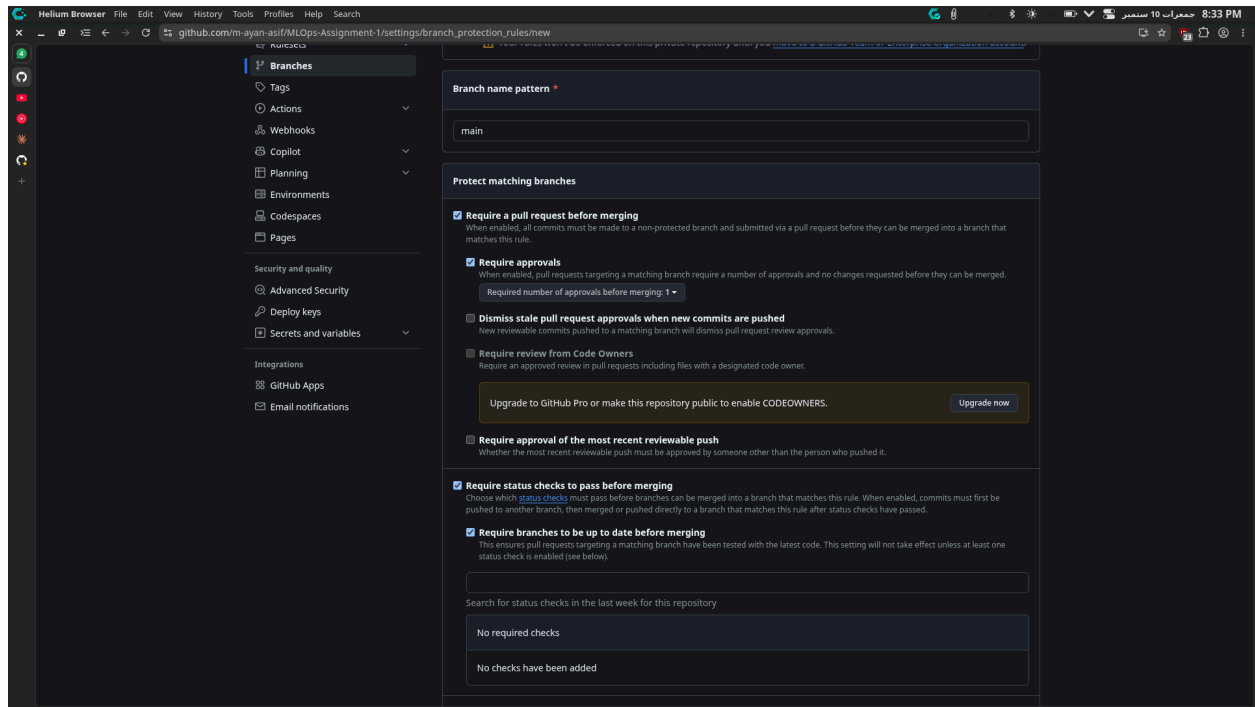
Assignment step:





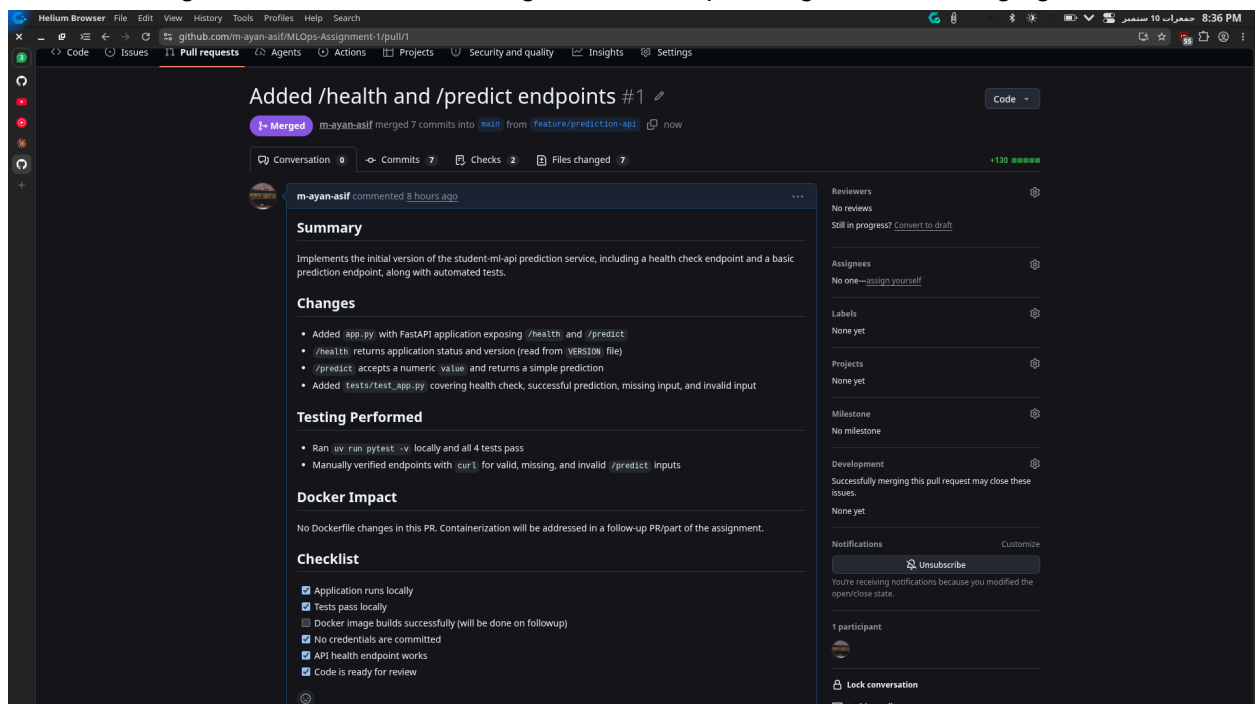
- Real CI failure: `test` job failed with `ModuleNotFoundError: No module named 'app'` due to missing `pytest pythonpath` config
- Real fix: added `[tool.pytest.ini_options]` with `pythonpath = ["."]` to `pyproject.toml`, committed as `fix: add pytest pythonpath config so app module resolves in CI` (commit `8ca41f7`)
- Pushed, CI re-ran, `test` job now passes
- `docker-build-check` is *still* failing (no Dockerfile yet) – this is expected and will get fixed properly in Part 9
- Also implemented assignment methodology of breaking and fixing assertion

Part 7:

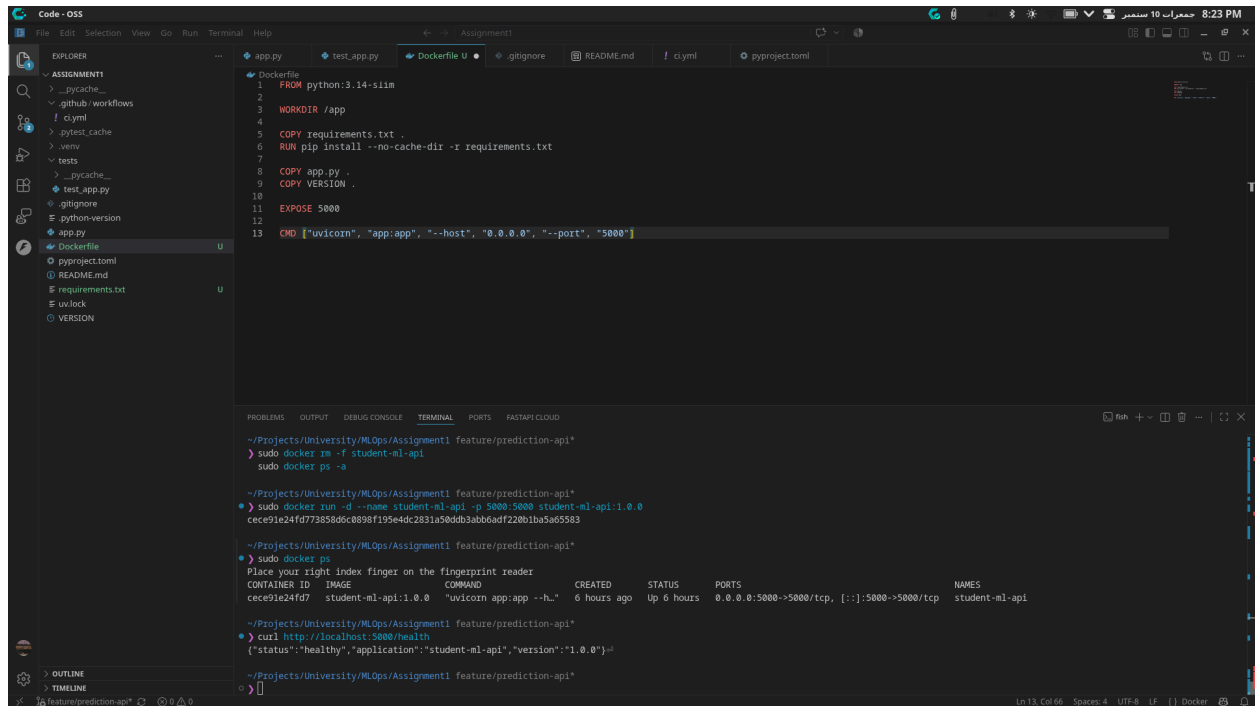


Part 8:

Chose to merge commit instead of hiding issues via squashing and then merging.



Part 9:



The screenshot shows the VS Code interface with the following content:

EXPLORER:

- ASSIGNMENT1
 - __pycache__
 - github/workflows
 - ! ci.yml
 - .pytest_cache
 - .venv
 - tests
 - __pycache__
 - test_app.py
 - .gitignore
 - python-version
 - app.py
 - Dockerfile
 - pyproject.toml
 - README.md
 - requirements.txt
 - uv.lock
 - VERSION

Dockerfile:

```
1 FROM python:3.14-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY app.py .
9 COPY VERSION .
10
11 EXPOSE 5000
12
13 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "5000"]
```

TERMINAL:

```
~/Projects/University/MLOps/Assignment1 feature/prediction-api*
> sudo docker rm -f student-ml-api
sudo docker ps -a

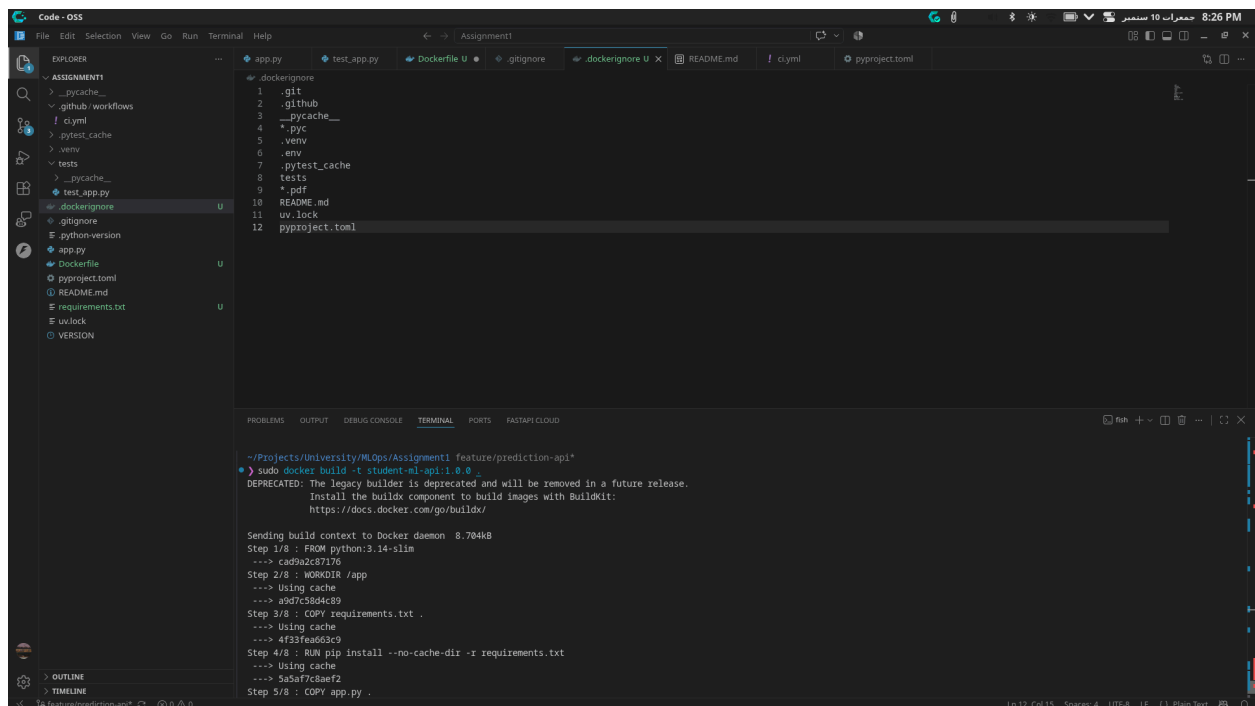
~/Projects/University/MLOps/Assignment1 feature/prediction-api*
> sudo docker run -d --name student-ml-api -p 5000:5000 student-ml-api:1.0.0
cece91e24fd773858dc8098f195e4dc2831a58ddh3abb6ad7228b1ba5a65583

~/Projects/University/MLOps/Assignment1 feature/prediction-api*
> sudo docker ps
Place your right index finger on the fingerprint reader
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
cece91e24fd7   student-ml-api:1.0.0                "uvicorn app:app --h..." 6 hours ago    Up 6 hours    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp    student-ml-api

~/Projects/University/MLOps/Assignment1 feature/prediction-api*
> curl http://localhost:5000/health
{"status":"healthy","application":"student-ml-api","version":"1.0.0"}

~/Projects/University/MLOps/Assignment1 feature/prediction-api*
> []
```

Part 10:



The screenshot shows the VS Code interface with the following content:

EXPLORER:

- ASSIGNMENT1
 - __pycache__
 - github/workflows
 - ! ci.yml
 - .pytest_cache
 - .venv
 - tests
 - __pycache__
 - test_app.py
 - .gitignore
 - python-version
 - app.py
 - Dockerfile
 - pyproject.toml
 - README.md
 - requirements.txt
 - uv.lock
 - VERSION

.dockerignore:

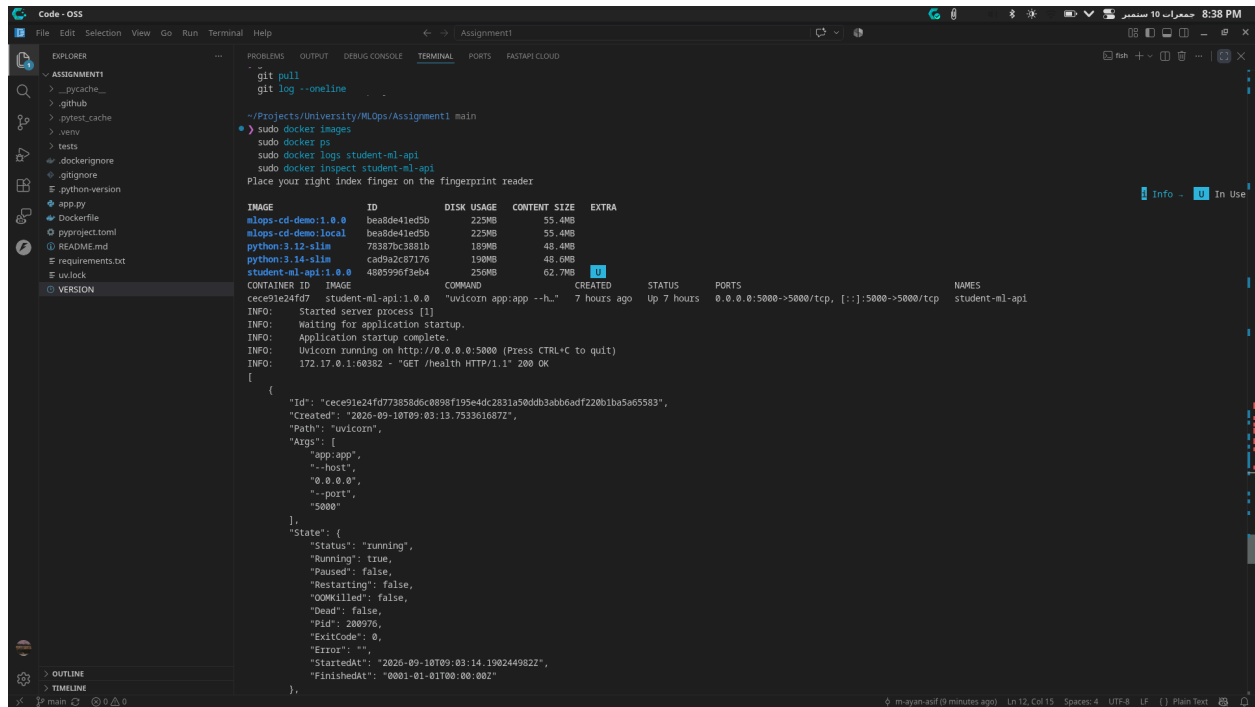
```
1 .git
2 .github
3 __pycache__
4 *.pyc
5 .venv
6 .env
7 .pytest_cache
8 tests
9 *.pdf
10 README.md
11 uv.lock
12 pyproject.toml
```

TERMINAL:

```
~/Projects/University/MLOps/Assignment1 feature/prediction-api*
> sudo docker build -t student-ml-api:1.0.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

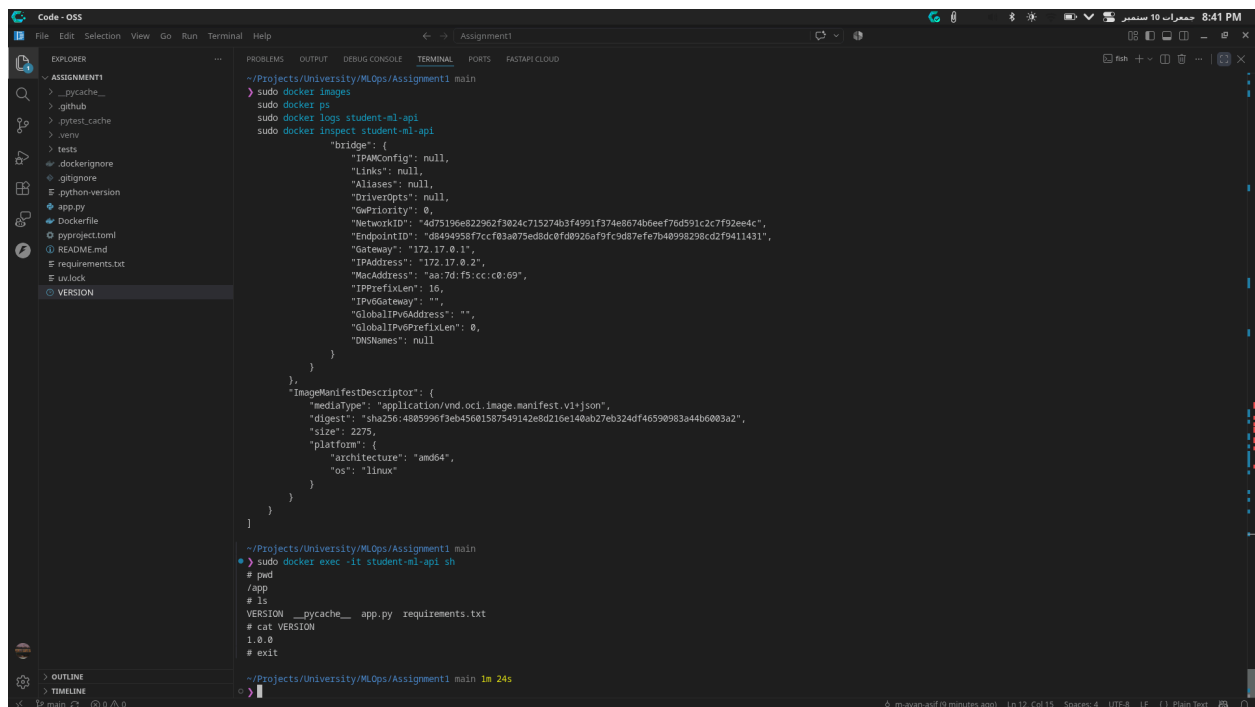
Sending build context to Docker daemon  8.704kB
Step 1/8 : FROM python:3.14-slim
--> c6d9a2c87176
Step 2/8 : WORKDIR /app
--> 4f33fead63c9
--> Using cache
--> a9d7c58d4c89
Step 3/8 : COPY requirements.txt .
--> Using cache
--> 4f33fead63c9
Step 4/8 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 5a5af7c8aef2
Step 5/8 : COPY app.py .
```

Part 11:



```
Code - OSS
File Edit Selection View Go Run Terminal Help
Assignment1
EXPLORER
ASSIGNMENT1
  __pycache__
  .github
  .pytest_cache
  .venv
  tests
  .dockerignore
  .gitignore
  app.py
  Dockerfile
  pyproject.toml
  README.md
  requirements.txt
  uv.lock
  VERSION
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS FASTAPI CLOUD
~/Projects/University/MLQps/Assignment1 main
git pull
git log --online
~/Projects/University/MLQps/Assignment1 main
sudo docker images
sudo docker ps
sudo docker logs student-ml-api
sudo docker inspect student-ml-api
Place your right index finger on the fingerprint reader

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
mlqps-cd-demo:1.0.0 bea8de41ed5b 225MB 55.4MB
mlqps-cd-demo:local bea8de41ed5b 225MB 55.4MB
python:3.12-slim 78387bc3881b 189MB 48.6MB
python:3.14-slim cad9a2c67176 190MB 48.6MB
student-ml-api:1.0.0 4805996f3eb4 250MB 62.7MB
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
cece91e24fd7 student-ml-api:1.0.0 "uvicorn app:app --h..." 7 hours ago Up 7 hours 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp student-ml-api
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
INFO: 172.17.0.1:60362 - "GET /health HTTP/1.1" 200 OK
{
  "Id": "cece91e24fd773858d6c0898f195e4dc2831a50ddb3abb6adf220b1ba5a65583",
  "Created": "2026-09-10T09:03:13.753361687Z",
  "Path": "uvicorn",
  "Args": [
    "app:app",
    "--host",
    "0.0.0.0",
    "--port",
    "5000"
  ],
  "State": {
    "Status": "running",
    "Running": true,
    "Paused": false,
    "Restarting": false,
    "OOMKilled": false,
    "Dead": false,
    "Pid": 280976,
    "ExitCode": 0,
    "Error": "",
    "StartedAt": "2026-09-10T09:03:14.190244982Z",
    "FinishedAt": "0001-01-01T00:00:00Z"
  },
```



```
Code - OSS
File Edit Selection View Go Run Terminal Help
Assignment1
EXPLORER
ASSIGNMENT1
  __pycache__
  .github
  .pytest_cache
  .venv
  tests
  .dockerignore
  .gitignore
  app.py
  Dockerfile
  pyproject.toml
  README.md
  requirements.txt
  uv.lock
  VERSION
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS FASTAPI CLOUD
~/Projects/University/MLQps/Assignment1 main
sudo docker images
sudo docker ps
sudo docker logs student-ml-api
sudo docker inspect student-ml-api
{
  "bridge": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": null,
    "DriverOpts": null,
    "GwPriority": 0,
    "NetworkID": "d475196e822962f3024c715274b3f4991f374e8674b6eef76d591c2cf92ee4c",
    "EndpointID": "d8494958f7ccf83a875ed8dc0fd0926af9fc9d87efe7b40998298cd2f9411431",
    "Gateway": "172.17.0.1",
    "IPAddress": "172.17.0.2",
    "MacAddress": "9a:7d:f5:cc:c0:69",
    "IPPrefixlen": 16,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6Prefixlen": 0,
    "DNSNames": null
  },
  "ImageManifestDescriptor": {
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "digest": "sha256:4805996f3eb43601587549142e80216e140ab27eb324df46598983a4b6003a2",
    "size": 2275,
    "platform": {
      "architecture": "amd64",
      "os": "linux"
    }
  }
}
~/Projects/University/MLQps/Assignment1 main
sudo docker exec -it student-ml-api sh
# pwd
/app
# ls
VERSION __pycache__ app.py requirements.txt
# cat VERSION
1.0.0
# exit
~/Projects/University/MLQps/Assignment1 main 1m 24s
```

Container ID:

cece91e24fd773858d6c0898f195e4dc2831a50ddb3abb6adf220b1ba5a65583 (short form: cece91e24fd7)

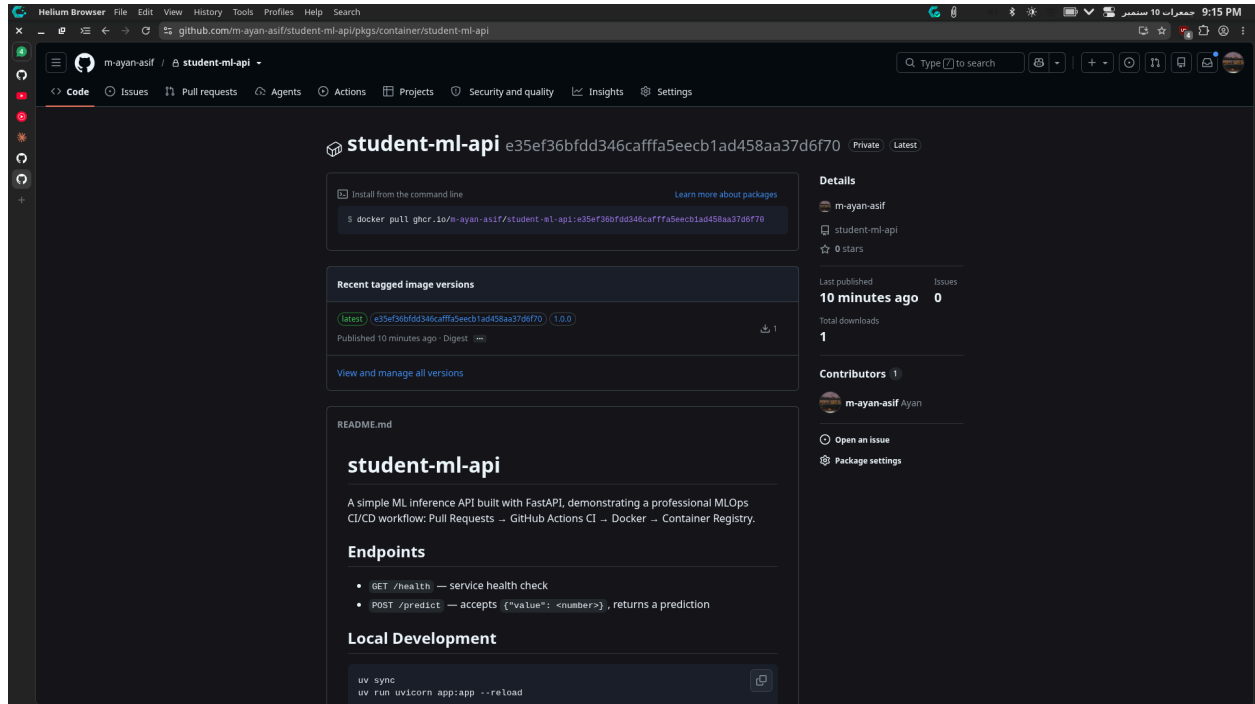
Image ID: 4805996f3eb4

Exposed port: 5000/tcp (mapped to host 5000)

Running command: `uvicorn app:app --host 0.0.0.0 --port 5000`

Application working directory: `/app`

Part 12:

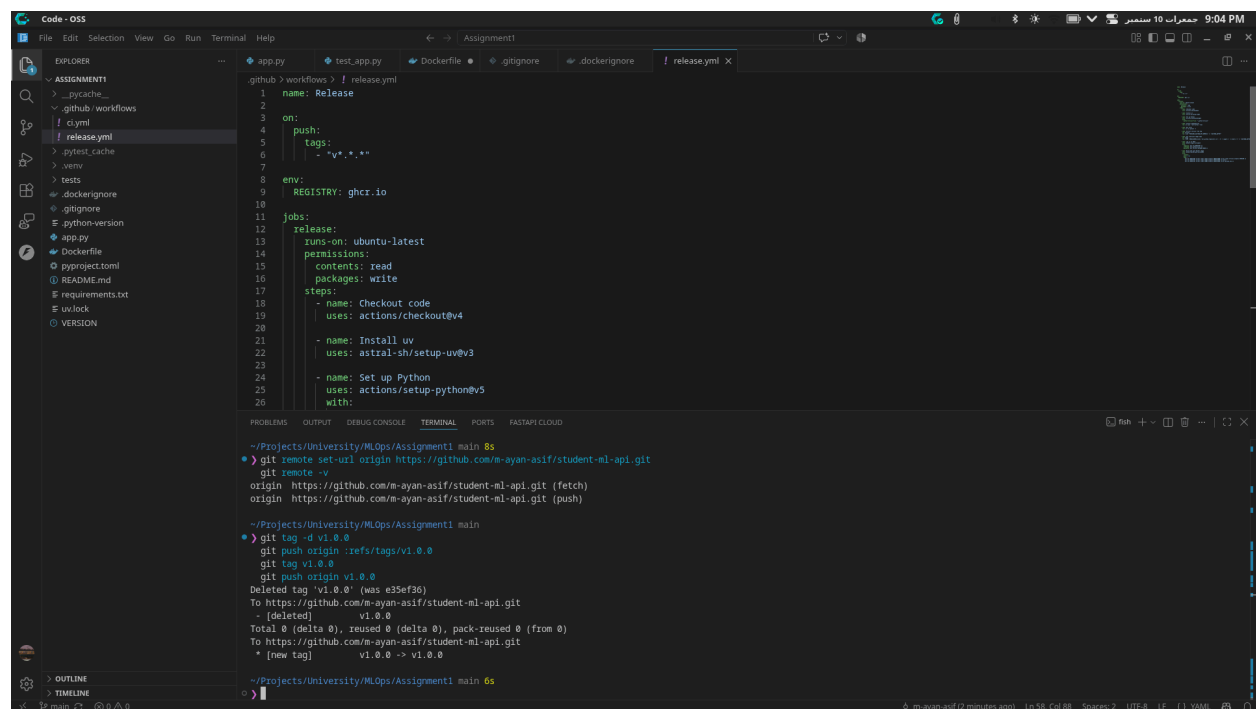


Live at ghcr.io/m-ayan-asif/student-ml-api:1.0.0

Part 13:

```
~/Projects/University/MLOps/Assignment1 main
• > git tag -d v1.0.0
git push origin :refs/tags/v1.0.0
git tag v1.0.0
git push origin v1.0.0
Deleted tag 'v1.0.0' (was e35ef36)
To https://github.com/m-ayan-asif/student-ml-api.git
- [deleted]          v1.0.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/m-ayan-asif/student-ml-api.git
* [new tag]          v1.0.0 -> v1.0.0
```

Part 14:



The screenshot shows the Visual Studio Code interface with a GitHub Actions workflow file named `release.yml` open in the editor. The workflow is configured for a 'Release' event and includes steps for checking out code, installing uv, and setting up Python. The terminal at the bottom shows the execution of git commands to delete and recreate the v1.0.0 tag.

```
github > workflows > / release.yml
1  name: Release
2
3  on:
4    push:
5      tags:
6        - "v*.*.*"
7
8  env:
9    REGISTRY: ghcr.io
10
11  jobs:
12    release:
13      runs-on: ubuntu-latest
14      permissions:
15        contents: read
16        packages: write
17      steps:
18        - name: Checkout code
19          uses: actions/checkout@v4
20
21        - name: Install uv
22          uses: astral-sh/setup-uv@v3
23
24        - name: Set up Python
25          uses: actions/setup-python@v5
26          with:
27            python-version: '3.11'
28
29  
```

```
~/Projects/University/MLOps/Assignment1 main 6s
• > git remote set-url origin https://github.com/m-ayan-asif/student-ml-api.git
git remote -v
origin https://github.com/m-ayan-asif/student-ml-api.git (fetch)
origin https://github.com/m-ayan-asif/student-ml-api.git (push)

~/Projects/University/MLOps/Assignment1 main
• > git tag -d v1.0.0
git push origin :refs/tags/v1.0.0
git tag v1.0.0
git push origin v1.0.0
Deleted tag 'v1.0.0' (was e35ef36)
To https://github.com/m-ayan-asif/student-ml-api.git
- [deleted]          v1.0.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/m-ayan-asif/student-ml-api.git
* [new tag]          v1.0.0 -> v1.0.0

~/Projects/University/MLOps/Assignment1 main 6s
• >
```

Part 15:

The screenshot shows a GitHub Actions workflow run for the repository 'm-ayan-asif / student-mi-api'. The workflow is named 'fix: restore missing workflow name and trigger #5'. The run was triggered by a push to the 'v1.0.0' branch and completed successfully. The 'release' job, which runs on 'push', took 58 seconds. Below the job details, there is a 'release summary' section containing a 'Docker Build summary'. This summary provides instructions on how to download the build record archive and import it into Docker Desktop's Builds view. It includes a link to the build record: 'm-ayan-asif-student-mi-api-NEYQ5K.dockerbuild' (19.64 KB - includes 1 build record). A table lists the build record details:

ID	Name	Status	Cached	Duration
NEYQ5K	student-mi-api	completed	0%	18s

Below the table, there is a section for 'Build inputs' and a note that the job summary was generated at run-time.

Part 16:

The screenshot shows the Docker Hub page for the repository 'm-ayan-asif / student-mi-api'. The 'Recent tagged image versions' section displays the 'latest' tag, which points to the image 'e35ef36bfd346caffa5eecb1ad458aa37d6f70' with version '1.0.0'. The image was published 10 minutes ago. The digest is shown as 'sha256:496dd52ed9f485438657e5f454fe1be355c412c774dcafd475623cd9f14322a'. A download icon with the number '1' is visible next to the digest.

Part 17:

```
~/Projects/University/MLOps/Assignment1 main 65
$ sudo docker pull ghcr.io/m-ayan-asif/student-ml-api:1.0.0
$ sudo docker run -d --name repro-test -p 5001:5000 ghcr.io/m-ayan-asif/student-ml-api:1.0.0
$ curl http://localhost:5001/health
1.0.0: Pulling from m-ayan-asif/student-ml-api
68382f284c79: Pull complete
e69bd93a893e: Pull complete
f30a080906ae: Pull complete
b3b3d421097f: Pull complete
e7637f578ab1: Pull complete
Digest: sha256:496dd52ed9f485438657e5f454fe1be355c412c774dcafd475623cd9f14322a
Status: Downloaded newer image for ghcr.io/m-ayan-asif/student-ml-api:1.0.0
ghcr.io/m-ayan-asif/student-ml-api:1.0.0
5f80ba03d894694c359c16a3bdf43e502432564a72fd68e5a48af8a575364eea
curl: (56) Recv failure: Connection reset by peer

~/Projects/University/MLOps/Assignment1 main 225
$ sudo docker ps -a
$ sudo docker logs repro-test
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
5f80ba03d894   ghcr.io/m-ayan-asif/student-ml-api:1.0.0   "uvicorn app:app --h..." 37 seconds ago Up 36 seconds 0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp   repro-test
cece91e24fd7   student-ml-api:1.0.0                     "uvicorn app:app --h..." 7 hours ago    Up 7 hours    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   student-ml-api

INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)

~/Projects/University/MLOps/Assignment1 main
$ curl http://localhost:5001/health
{"status": "healthy", "application": "student-ml-api", "version": "1.0.0"}

~/Projects/University/MLOps/Assignment1 main
$ []
```

Part 18:

```
Code - OSS
File Edit Selection View Go Run Terminal Help
Assignment1
EXPLORER
  ASSIGNMENT1
    __pycache__
    .github/workflows
    .cylm
    release.yml
    .pytest_cache
    .venv
    tests
    __pycache__
    test_app.py
    Dockerfile
    .gitignore
    .python-version
    app.py
    Dockerfile
    pyproject.toml
    README.md
    requirements.txt
    uv.lock
    VERSION
  app.py
  Dockerfile
  pyproject.toml
  README.md
  requirements.txt
  uv.lock
  VERSION

app.py
8
9
10 class PredictRequest(BaseModel):
11     value: float
12
13
14 @app.get("/")
15 async def root():
16     return {"message": "Hello World"}
17
18
19 @app.get("/health")
20 async def health():
21     return {
22         "status": "healthy",
23         "application": "student-ml-api",
24         "application_version": VERSION,
25         "model_version": "model-1"
26     }
27
28
29 @app.post("/predict")
30 async def predict(req: PredictRequest):
31     prediction = req.value * 2
32     return {"input": req.value, "prediction": prediction}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS FASTAPI CLOUD
~/Projects/University/MLOps/Assignment1 feature/model-metadata*
$ uv run pytest -v
tests/test_app.py::test_predict_invalid_input PASSED [100%]

===== warnings summary =====
venv/lib/python3.14/site-packages/fastapi/testclient.py:1
/home/ayan/Projects/University/MLOps/Assignment1/venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

venv/lib/python3.14/site-packages/starlette/testclient.py:53
/home/ayan/Projects/University/MLOps/Assignment1/venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 4 passed, 2 warnings in 0.38s =====

~/Projects/University/MLOps/Assignment1 feature/model-metadata*
$ []
```

```
6 def test_health():
7     response = client.get("/health")
8     assert response.status_code == 200
9     body = response.json()
10    assert body["status"] == "healthy"
11    assert body["application"] == "student-ml-api"
12    assert body["application_version"] == "1.1.0"
13    assert body["model_version"] == "model-1"
14
15
16 def test_predict_success():
17     response = client.post("/predict", json={"value": 10})
18     assert response.status_code == 200
19     body = response.json()
20     assert body["input"] == 10.0
21     assert body["prediction"] == 20.0
22
23
24 def test_predict_missing_input():
25     response = client.post("/predict", json={})
26     assert response.status_code == 422
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS FASTAPI CLOUD

~/Projects/University/MLOps/Assignment1 feature/model-metadata*

> uv run pytest -v

tests/test_app.py::test_predict_invalid_input PASSED [100%]

===== warnings summary =====

.venv/lib/python3.14/site-packages/fastapi/testclient.py:1
/home/ayan/Projects/University/MLOps/Assignment1/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
 from starlette.testclient import TestClient as TestClient # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53
/home/ayan/Projects/University/MLOps/Assignment1/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio._PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

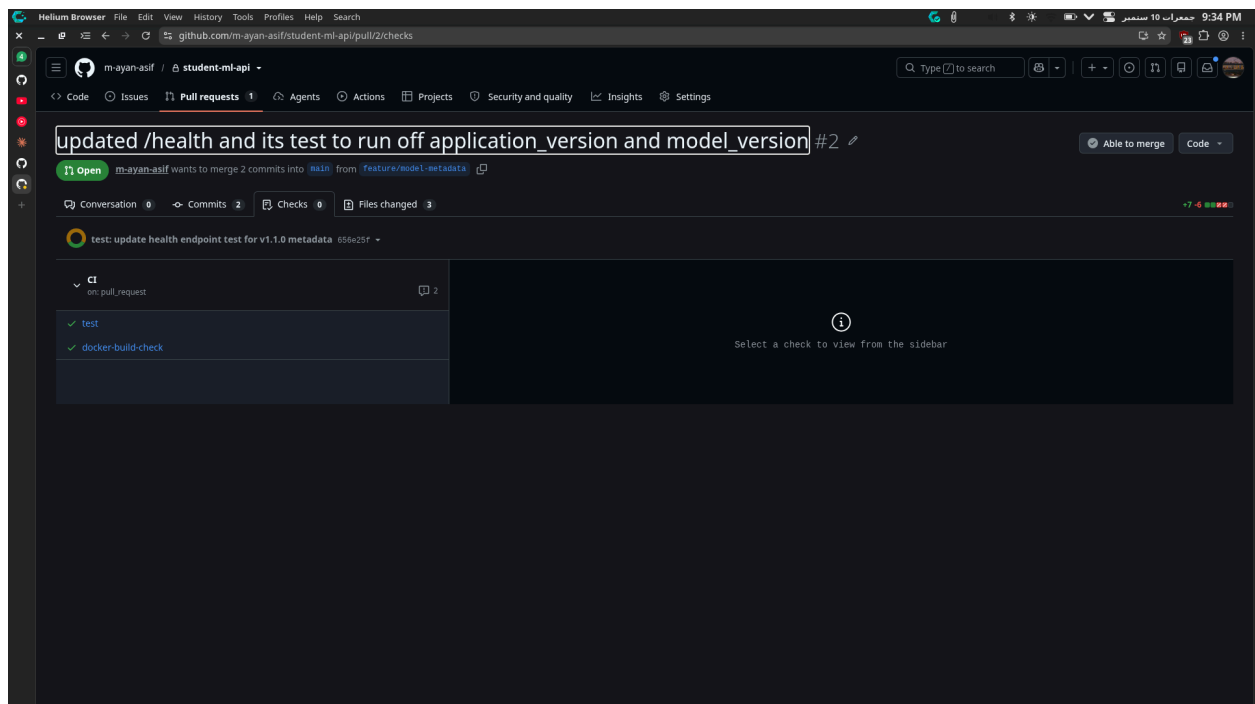
===== 4 passed, 2 warnings in 0.38s =====

~/Projects/University/MLOps/Assignment1 feature/model-metadata*

>

Ln 14, Col 5 Spaces: 4 UTF-8 LF Python

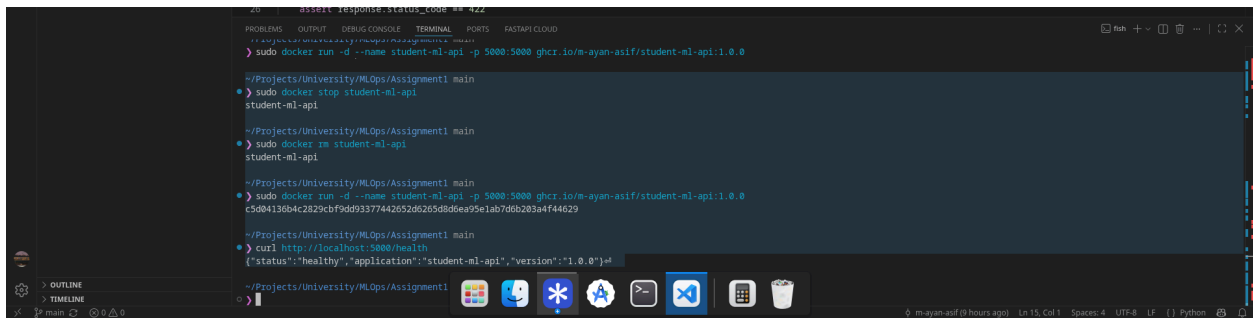
Part 19:



```
~/Projects/University/MLOps/Assignment1 main
• > git tag -l
  git log --oneline -3 v1.1.0
  v1.0.0
  v1.1.0
  99d6f4e (HEAD -> main, tag: v1.1.0, origin/main, origin/HEAD) Merge pull request #2 from m-ayan-asif/feature/model-metadata
  656e25f (origin/feature/model-metadata, feature/model-metadata) test: update health endpoint test for v1.1.0 metadata
  515aa3c feat: add model version metadata to health endpoint

~/Projects/University/MLOps/Assignment1 main
```

Part 20:



```
~/Projects/University/MLOps/Assignment1 main
• > sudo docker run -d --name student-ml-api -p 5000:5000 ghcr.io/m-ayan-asif/student-ml-api:1.0.0

~/Projects/University/MLOps/Assignment1 main
• > sudo docker stop student-ml-api
student-ml-api

~/Projects/University/MLOps/Assignment1 main
• > sudo docker rm student-ml-api
student-ml-api

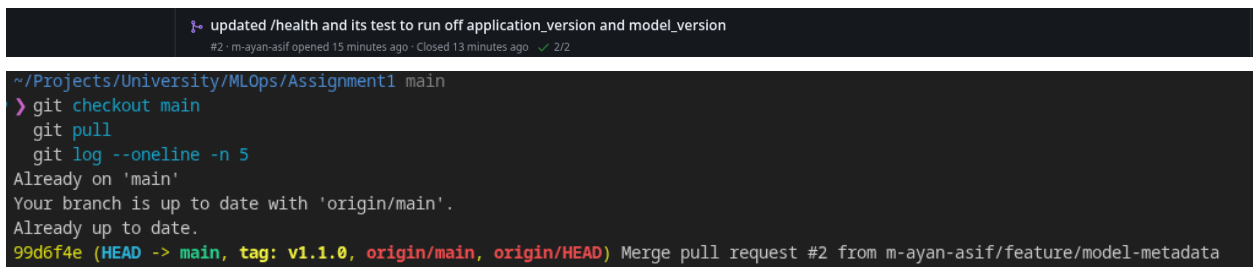
~/Projects/University/MLOps/Assignment1 main
• > sudo docker run -d --name student-ml-api -p 5000:5000 ghcr.io/m-ayan-asif/student-ml-api:1.0.0
c5d8413b84c2829cbf9dd93377442652d6265d8d6ea95e1ab7d8b283a4f44629

~/Projects/University/MLOps/Assignment1 main
• > curl http://localhost:5000/health
{"status": "healthy", "application": "student-ml-api", "version": "1.0.0"}

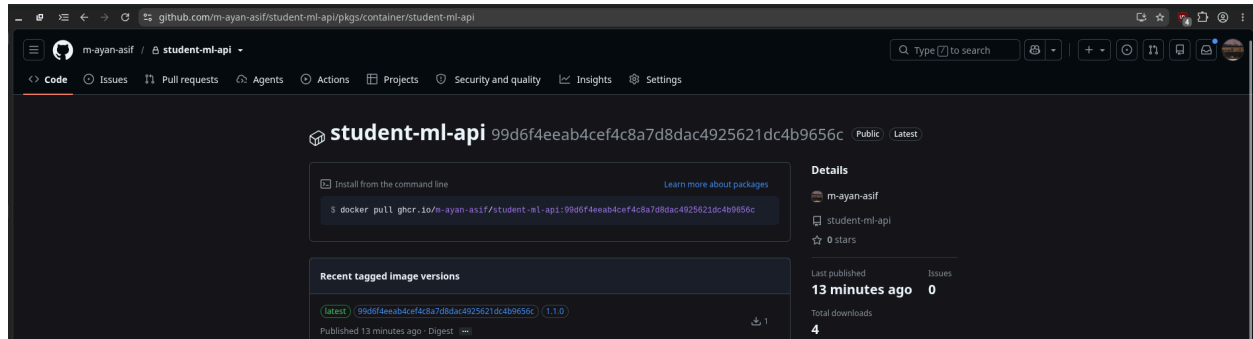
~/Projects/University/MLOps/Assignment1
```

Rolling back using a container registry is instantaneous and reliable because the **1.0.0** Docker image is an immutable, frozen snapshot. The OS, dependencies, and code are exactly as they were when it was originally built. If we used **git clone** and **pip install** today, we would risk dependency drift (fetching newer sub-dependencies that might break the app) and we would have to wait for the entire build process to run again, risking build-time network failures.

Part 21:



```
~/Projects/University/MLOps/Assignment1 main
• > git checkout main
  git pull
  git log --oneline -n 5
  Already on 'main'
  Your branch is up to date with 'origin/main'.
  Already up to date.
  99d6f4e (HEAD -> main, tag: v1.1.0, origin/main, origin/HEAD) Merge pull request #2 from m-ayan-asif/feature/model-metadata
```



PR: | #2

Merge Commit: | 99d6f4e

Git Tag: | v1.1.0

Docker Image: | student-ml-api:1.1.0

Image Digest: |

sha256:d6b87f23cabfcfaaabc1796e3b57ef68fdb7d83623facd0b1237fc2a50b60ab7

Part 22:

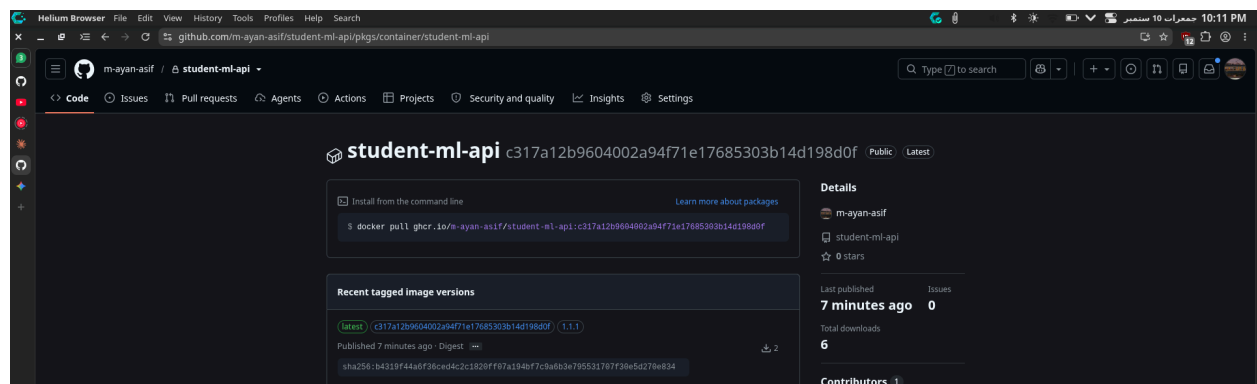
Publishing Docker images directly from every Pull Request is undesirable for several key reasons:

- **Registry Bloat and Storage Costs:** Pull Requests often receive frequent commits. Building and pushing an image for every single update rapidly consumes container registry storage.
- **Artifact Untrustworthiness:** Code within a Pull Request is inherently experimental, actively under review, and potentially broken. A container registry should serve as a trusted, stable repository of deployable release artifacts.
- **Security Vulnerabilities:** Publishing an image requires the CI pipeline to use write-access credentials for the container registry. Exposing these highly privileged secrets to PR workflows increases the risk of credential theft or abuse, particularly in environments that accept external contributions.
- **Versioning Obfuscation:** Flooding the registry with hundreds of temporary PR images clutters the version history. It obscures the actual stable, semantic releases (like **1.0.0** or **1.1.0**), making artifact traceability, deployment management, and rollbacks significantly more difficult.

Part 23:

```
~/Projects/University/MLOps/Assignment1 main
* sudo docker pull ghcr.io/m-ayan-asif/student-ml-api:1.1.1
  sudo docker inspect ghcr.io/m-ayan-asif/student-ml-api:1.1.1 --format='{{json .Config.Labels}}'
1.1.1: Pulling from m-ayan-asif/student-ml-api
b612a8fac3ad: Pull complete
539cb3465db2: Pull complete
9ffbd9fce838: Pull complete
80c9de210feb: Pull complete
5e51623234e7: Pull complete
Digest: sha256:b4319f44a6f36ced4c2c1820ff07a194bf7c9a6b3e795531707f30e5d270e834
Status: Downloaded newer image for ghcr.io/m-ayan-asif/student-ml-api:1.1.1
ghcr.io/m-ayan-asif/student-ml-api:1.1.1
{"org.opencontainers.image.created":"2026-09-10T17:03:14Z","org.opencontainers.image.revision":"c317a12b9604002a94f71e17685303b14d198d0f","org.opencontainers.image.source":"https://github.com/m-ayan-asif/student-ml-api","org.opencontainers.image.title":"student-ml-api","org.opencontainers.image.version":"1.1.1"}
```

Part 24:



The screenshot shows the GitHub repository page for `student-ml-api` by user `m-ayan-asif`. The repository is public and has a commit hash of `c317a12b9604002a94f71e17685303b14d198d0f`. The page displays the following information:

- Install from the command line:** A code block showing the command `docker pull ghcr.io/m-ayan-asif/student-ml-api:c317a12b9604002a94f71e17685303b14d198d0f`.
- Recent tagged image versions:** A table showing the latest version `1.1.1` with a commit hash of `c317a12b9604002a94f71e17685303b14d198d0f`. It also shows the SHA256 digest `sha256:b4319f44a6f36ced4c2c1820ff07a194bf7c9a6b3e795531707f30e5d270e834`.
- Details:** The repository has 0 stars, 0 issues, and 6 total downloads. It was last published 7 minutes ago.

Absolute Traceability: Semantic versions (like `1.1.0`) are great for humans, but they can technically be overwritten or deleted. A commit SHA tag provides an immutable, unbreakable link between the Docker image running in production and the exact state of the source code repository. If an error occurs, you don't have to guess which version of the code was used to build that image because the tag tells you the exact Git commit, making debugging and auditing significantly easier.

Part 25:

```

12
13 #cache test 1
14 @app.get("/")
15 async def root():
16     return {"message": "Hello World"}
17
18
19 @app.get("/health")
20 async def health():
21     return {
22         "status": "healthy",
23         "application": "student-ml-api",
24         "application_version": VERSION,
25         "model_version": "model-1"
26     }
27
28
29 @app.post("/predict")
30 async def predict(req: PredictRequest):
31     prediction = req.value * 2
32     return {"input": req.value, "prediction": prediction}

```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS FASTAPI CLOUD
```

```
~/Projects/University/MLops/Assignment1 main lm 14s
$ sudo docker build -t cache-test .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  9.216kB
Step 1/13 : FROM python:3.14-slim
--> cad9a2c87176
Step 2/13 : ARG BUILD_DATE
--> Using cache
--> 8c0c261dc5bc
Step 3/13 : ARG GIT_COMMIT
--> Using cache
```

```

tests > test_app.py
1 from fastapi.testclient import TestClient
2 from app import app
3
4 client = TestClient(app)
5
6 def test_health():
7     response = client.get("/health")
8     assert response.status_code == 200
9     body = response.json()
10    assert body["status"] == "healthy"
11    assert body["application"] == "student-ml-api"
12    assert body["application_version"] == "1.1.0"
13    assert body["model_version"] == "model-1"
14
15 #cache test 2
16 def test_predict_success():
17     response = client.post("/predict", json={"value": 10})
18     assert response.status_code == 200
19     body = response.json()
20     assert body["input"] == 10.0
21     assert body["prediction"] == 20.0
22
23
24 def test_predict_missing_input():
25     response = client.post("/predict", json={})
26     assert response.status_code == 422

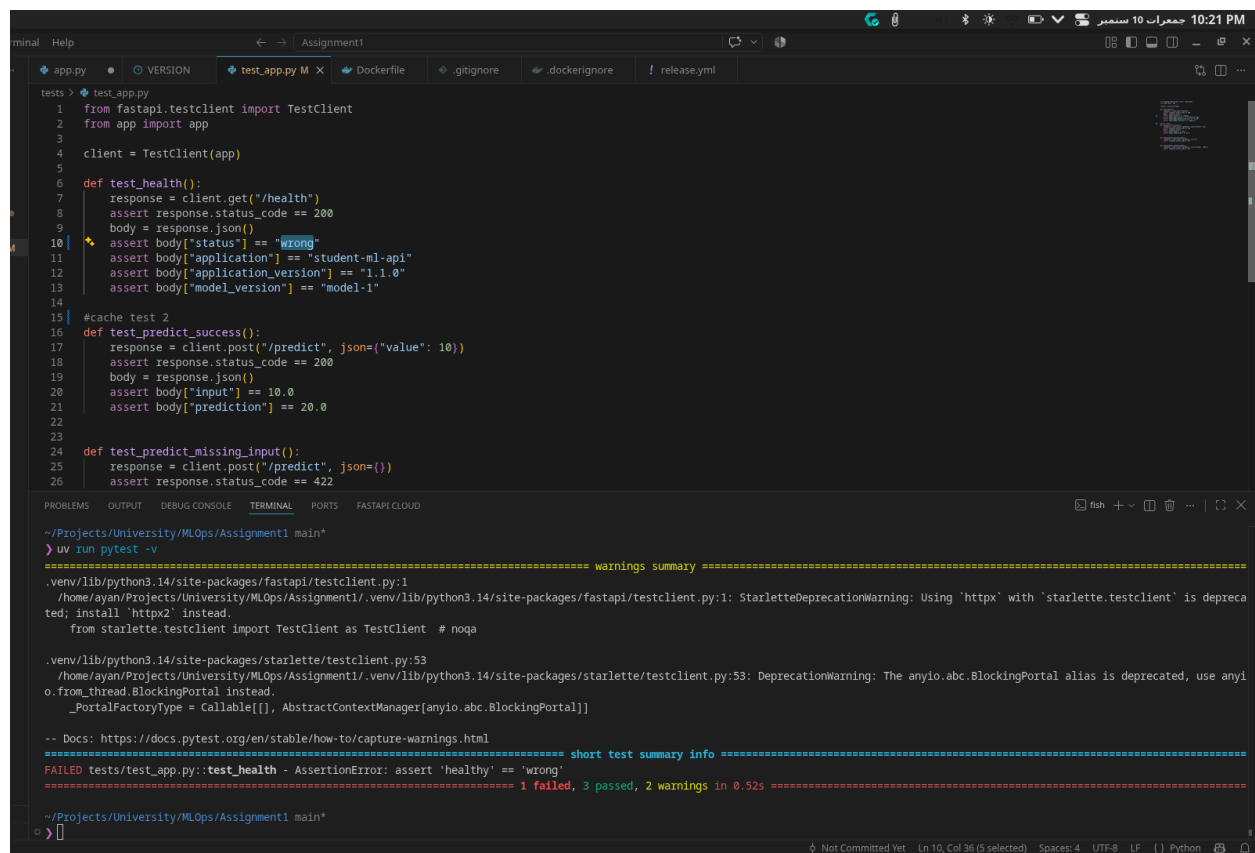
```

How Layers Work: Docker builds images sequentially, with each instruction (**COPY**, **RUN**) creating a new layer. Docker caches these layers to speed up future builds. However, if a file in a **COPY** layer changes, Docker invalidates the cache for that layer and every layer after it as told in the class on 8th September.

The Efficient Way: Source code (**app.py**) changes constantly during development, while dependencies (**requirements.txt**) change rarely. By executing **COPY requirements.txt .** and **RUN pip install -r requirements.txt** first, you lock that heavy installation step into the cache. When you subsequently **COPY app.py .**, modifying your code only invalidates the final code-copy layer, leaving the heavy **pip install** layer cached and making the build nearly instantaneous.

The Inefficient Way: If you use **COPY . .** before **RUN pip install**, any minor code edit (like fixing a typo) invalidates the very first layer. This completely destroys the cache, forcing Docker to unnecessarily re-download and re-install every single Python package on every single build, wasting massive amounts of time and network bandwidth.

Part 26:



```
tests > test_app.py
1 from fastapi.testclient import TestClient
2 from app import app
3
4 client = TestClient(app)
5
6 def test_health():
7     response = client.get("/health")
8     assert response.status_code == 200
9     body = response.json()
10    ✖ assert body["status"] == "wrong"
11    assert body["application"] == "student-ml-api"
12    assert body["application_version"] == "1.1.0"
13    assert body["model_version"] == "model-1"
14
15 #cache test 2
16 def test_predict_success():
17     response = client.post("/predict", json={"value": 10})
18     assert response.status_code == 200
19     body = response.json()
20     assert body["input"] == 10.0
21     assert body["prediction"] == 20.0
22
23
24 def test_predict_missing_input():
25     response = client.post("/predict", json={})
26     assert response.status_code == 422

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS FASTAPI CLOUD
~/Projects/University/MLOps/Assignment1 main*
> uv run pytest -v

===== warnings summary =====
.../lib/python3.14/site-packages/fastapi/testclient.py:1
/home/ayan/Projects/University/MLOps/Assignment1/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated, install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa
.../lib/python3.14/site-packages/starlette/testclient.py:53
/home/ayan/Projects/University/MLOps/Assignment1/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== short test summary info =====
FAILED tests/test_app.py::test_health - AssertionError: assert 'healthy' == 'wrong'
===== 1 failed, 3 passed, 2 warnings in 0.52s =====

~/Projects/University/MLOps/Assignment1 main*
o >
```

- **Symptom:** The automated test suite fails and exits with a non-zero status code during local execution or CI.
- **Root Cause:** A unit test assertion explicitly expects a value ("wrong") that does not match the actual JSON output of the application logic ("healthy").
- **Correction:** Reviewed the failed test output, identified the mismatch, and corrected the assertion in `test_app.py` to match the expected application behavior.

```

24 EXPOSE 5000
25
26 CMD ["uvicorn", "app:app", "--host", "127.0.0.1", "--port", "5000"]

```

```

~/Projects/University/MLOps/Assignment1 main*
> sudo docker build -t fail-test .
--> Using cache
--> e16144af5050
Step 13/13 : CMD ["uvicorn", "app:app", "--host", "127.0.0.1", "--port", "5000"]
--> Using cache
--> 520ba5da22cf
Successfully built 520ba5da22cf
Successfully tagged fail-test:latest

~/Projects/University/MLOps/Assignment1 main*
> sudo docker run -d -p 5000:5000 --name fail-container fail-test
d41b5f2b24e4108f6bce3f902f12690f2213b355a1a49c04be85700160d0028

~/Projects/University/MLOps/Assignment1 main*
> curl http://localhost:5000/health
curl: (56) Recv failure: Connection reset by peer

~/Projects/University/MLOps/Assignment1 main*
>

```

Symptom: The Docker container starts successfully, but the API is completely unreachable from the host machine via `curl`.

Root Cause: The Uvicorn server inside the container is instructed to listen only on the internal loopback interface (`127.0.0.1`). It ignores all external traffic routed by Docker, rendering the `-p 5000:5000` port mapping useless.

Correction: Modified the application startup command in the `Dockerfile` to bind to `--host 0.0.0.0`. This forces the server to listen on all network interfaces, allowing Docker to route external traffic to the app